

Trabajo Práctico 2: Git y GitHub

Objetivo:

El estudiante desarrollará competencias para trabajar con Git y GitHub, aplicando conceptos fundamentales de control de versiones, colaboración en proyectos y resolución de conflictos, en un entorno simulado y guiado.

Resultados de aprendizaje:

1. Comprender los conceptos básicos de Git y GitHub: Identificar y explicar los principales términos y procesos asociados con Git y GitHub, como repositorios, ramas, commits, forks, etiquetas y repositorios remotos.
2. Manejar comandos esenciales de Git: Ejecutar comandos básicos para crear, modificar, fusionar y gestionar ramas, commits y repositorios, tanto en local como en remoto.
3. Aplicar técnicas de colaboración en GitHub: Configurar y utilizar repositorios remotos, realizar forks, y gestionar pull requests para facilitar el trabajo colaborativo.
4. Resolver conflictos en un entorno de control de versiones: Identificar, analizar y solucionar conflictos de merge generados en un flujo de trabajo con múltiples ramas.

Actividades

- 1) Contestar las siguientes preguntas utilizando las guías y documentación proporcionada (Desarrollar las respuestas) :

- ¿Qué es GitHub?

Es un sitio para alojar proyectos utilizando el sistema de control de GIT.

- ¿Cómo crear un repositorio en GitHub?

Primero hay que crear una cuenta, luego en nuestro proyecto para iniciar le damos al comando `git init`, luego `git add .`, luego `git commit -m "y lo que se haya agregado/corregido"`, y por último `git push origin master` (o la rama que se esté usando). Cuando entramos a GitHub ya tendremos nuestro repositorio subido.

- ¿Cómo crear una rama en Git?

`git branch nombre-rama`

- ¿Cómo cambiar a una rama en Git?

`git checkout`

- ¿Cómo fusionar ramas en Git?

`git merge`

- ¿Cómo crear un commit en Git?

`git commit -m "El comentario de lo agregado/corregido"`

- ¿Cómo enviar un commit a GitHub?

`Git commit -m "mensaje"`

- ¿Qué es un repositorio remoto?

Es una copia de un proyecto alojada en un servidor remoto como internet.

- ¿Cómo agregar un repositorio remoto a Git?

`git remote add origin url`

- ¿Cómo empujar cambios a un repositorio remoto?

`git push -u origin master`

- ¿Cómo tirar de cambios de un repositorio remoto?

`git pull`

- ¿Qué es un fork de repositorio?

Es una copia exacta de un repositorio original.

- ¿Cómo crear un fork de un repositorio?

Ir a la página donde se quiere copiar el fork, dar click en el botón a la derecha que dice fork, cambiar, o no el nombre de la descripción, hacer click en Create Fork y listo, ya tenes una copia del fork.

- ¿Cómo enviar una solicitud de extracción (pull request) a un repositorio?

Hay que ir a la página principal del repositorio, seleccionar la rama que tiene cambios, hacer click en "Comparar y solicitud de incorporación de cambios", elegir la rama base y la rama de comparación, escribir un título y una descripción, hacer clic en "Crear solicitud de incorporación de cambios" y esperar la respuesta.

- ¿Cómo aceptar una solicitud de extracción?

Hay que hacer clic en Solicitudes de incorporación de cambios en el nombre del repositorio, seleccionar la solicitud de extracción que se quiere revisar, hacer clic en revisar cambios sobre el código modificado, escribir un comentario sobre los cambios, hacer clic en aprobar, luego en enviar revisión, después en fusionar solicitud de extracción, añadir nombre y descripción, y confirmar la fusión.

- ¿Qué es una etiqueta en Git?
Es una referencia que va a puntos concretos en el historial de Git.
- ¿Cómo crear una etiqueta en Git?
Insertar el comando git tag y nombre de la etiqueta
- ¿Cómo enviar una etiqueta a GitHub?
Luego de crear una etiqueta, usar el comando git push nombre_etiqueta.
- ¿Qué es un historial de Git?
Es un registro donde se pueden ver los cambios realizados del código en un repositorio.
- ¿Cómo ver el historial de Git?
Con el comando git log.
- ¿Cómo buscar en el historial de Git?
Con el comando git log --oneline (ver los commits por línea)
- ¿Cómo borrar el historial de Git?
Comando git clean.
- ¿Qué es un repositorio privado en GitHub?
Es un lugar donde se almacena archivos, código y revisiones, pero de forma privada para que otros lo vean tenes que otorgar permiso.
- ¿Cómo crear un repositorio privado en GitHub?
De la misma manera como creamos un repositorio público, pero a la hora de elegir la visibilidad, escoges que sea privado.
- ¿Cómo invitar a alguien a un repositorio privado en GitHub?
Para invitar a alguien a ver el repositorio privado hay que ir a configuración, donde dice colaboradores y equipos, y agregar personas. Desde ahí se administra el acceso al repositorio ingresando su correo electrónico o nombre de usuario.
- ¿Qué es un repositorio público en GitHub?
Es un espacio en el que se puede compartir y almacenar código de forma accesible a todos los usuarios.
- ¿Cómo crear un repositorio público en GitHub?

Hay que iniciar sesión en GitHub, ir a seleccionar nuevo repositorio, elegir el nombre, agregar descripción y en visibilidad ponerlo público.

- ¿Cómo compartir un repositorio público en GitHub?

En la página donde esta alojado el repositorio hay un botón verde donde dice Code, desde allí se puede copiar y compartir el enlace a nuestro repositorio.

2) Realizar la siguiente actividad:

- Crear un repositorio.
 - Dale un nombre al repositorio.
 - Elije el repositorio sea público.
 - Inicializa el repositorio con un archivo.
- Agregando un Archivo
 - Crea un archivo simple, por ejemplo, "mi-archivo.txt".
 - Realiza los comandos `git add .` y `git commit -m "Agregando mi-archivo.txt"` en la línea de comandos.
 - Sube los cambios al repositorio en GitHub con `git push origin main` (o el nombre de la rama correspondiente).
- Creando Branchs
 - Crear una Branch
 - Realizar cambios o agregar un archivo
 - Subir la Branch

3) Realizar la siguiente actividad:

Paso 1: Crear un repositorio en GitHub

- Ve a GitHub e inicia sesión en tu cuenta.

- Haz clic en el botón "New" o "Create repository" para crear un nuevo repositorio.
- Asigna un nombre al repositorio, por ejemplo, conflict-exercise.
- Opcionalmente, añade una descripción.
- Marca la opción "Initialize this repository with a README".
- Haz clic en "Create repository".

Paso 2: Clonar el repositorio a tu máquina local

- Copia la URL del repositorio (usualmente algo como `https://github.com/tuusuario/conflict-exercise.git`).
- Abre la terminal o línea de comandos en tu máquina.
- Clona el repositorio usando el comando:

```
git clone https://github.com/tuusuario/conflict-exercise.git
```

- Entra en el directorio del repositorio: `cd conflict-exercise`

Paso 3: Crear una nueva rama y editar un archivo

- Crea una nueva rama llamada feature-branch:

```
git checkout -b feature-branch
```

- Abre el archivo README.md en un editor de texto y añade una línea nueva, por ejemplo:

Este es un cambio en la feature branch.

- Guarda los cambios y haz un commit: `git add README.md` `git commit -m "Added a line in feature-branch"`

Paso 4: Volver a la rama principal y editar el mismo archivo

- Cambia de vuelta a la rama principal (main):

```
git checkout main
```

- Edita el archivo README.md de nuevo, añadiendo una línea diferente:

Este es un cambio en la main branch.

- Guarda los cambios y haz un commit: `git add README.md` `git commit -m "Added a line in main branch"`

Paso 5: Hacer un merge y generar un conflicto

- Intenta hacer un merge de la feature-branch en la rama main:

`git merge feature-branch`

- Se generará un conflicto porque ambos cambios afectan la misma línea del archivo README.md.

Paso 6: Resolver el conflicto

- Abre el archivo README.md en tu editor de texto. Verás algo similar a esto:

```
<<<<<< HEAD
```

Este es un cambio en la main branch.

```
=====
```

Este es un cambio en la feature branch.

```
>>>>>> feature-branch
```

- Decide cómo resolver el conflicto. Puedes mantener ambos cambios, elegir uno de ellos, o fusionar los contenidos de alguna manera.
- Edita el archivo para resolver el conflicto y guarda los cambios (Se debe borrar lo marcado en verde en el archivo donde estes solucionando el conflicto. Y se debe borrar la parte del texto que no se quiera dejar).
- Añade el archivo resuelto y completa el merge:

```
git add README.md git commit -m
```

```
"Resolved merge conflict"
```

Paso 7: Subir los cambios a GitHub

- Sube los cambios de la rama main al repositorio remoto en GitHub:

```
git push origin main
```

- También sube la feature-branch si deseas:

```
git push origin feature-branch
```

Paso 8: Verificar en GitHub

- Ve a tu repositorio en GitHub y revisa el archivo README.md para confirmar que los cambios se han subido correctamente.
- Puedes revisar el historial de commits para ver el conflicto y su resolución.

Camila Cynthia Reyna – Comisión 20.

Agrego el link de mi repositorio para mostrar la actividad práctica con GIT y GITHUB.